MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on laboratory work no. 1

of Cryptography and Security

Topic: The Caesar Cipher and the Caesar Cipher with a Permutation

Done by: st. gr. FAF-243, Augustin Carazanu
Checked by: *assist. univ. Maia Zaica*
FCIM, UTM

Chişinău – 2026

Purpose of the work

The purpose of this laboratory work is to understand how the Caesar cipher works, to see why its small key space makes it insecure, to strengthen it with a keyword-based permutation of the alphabet, and to implement both versions for the 31-letter Romanian alphabet. Both tasks were implemented in Python 3 as one program with a simple graphical interface built with Tkinter.

1. Short theory

1.1. The Caesar cipher

The Caesar cipher is a monoalphabetic substitution cipher: every letter of the plaintext is replaced by the letter located a fixed number of positions k further along the alphabet, wrapping around at the end. Each letter is represented by its numeric code $x \in \{0, 1, \ldots, n-1\}$, where n is the length of the alphabet, and the same secret key k is used for both operations:

$$c = e_k(x) = (x + k) \bmod n$$

$$m = d_k(y) = (y - k) \bmod n$$

When $y - k$ is negative, the result is brought back into the range $0 \ldots n-1$ by adding n. For the Romanian alphabet n = 31 and the letters are encoded as in Table 1.1.

Table 1.1 – Letter encoding of the Romanian alphabet (n = 31)

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
Code	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Letter	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z
Code	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

Key space. The key can take the values 1, 2, …, n − 1 (k = 0 leaves the text unchanged), so over the Romanian alphabet there are only **30 possible keys**. An attacker can simply try all of them (brute-force attack): exactly one key gives readable text.

1.2. The Caesar cipher with a permutation

To enlarge the key space, a second key, the keyword k_2, is used to reorder the alphabet. The permuted alphabet is built by writing the distinct letters of the keyword first (only the first occurrence of each letter is kept), followed by the remaining letters in their natural order. The shift k_1 is then applied to the positions in this new alphabet:

$$c = \pi[(\pi^{-1}(x) + k_1) \bmod n], \quad m = \pi[(\pi^{-1}(y) - k_1) \bmod n]$$

where π is the permuted alphabet and $\pi^{-1}(x)$ is the position of letter x in it. For $k_1 = 3$ and k_2 = CRIPTOGRAFIE the distinct keyword letters are C, R, I, P, T, O, G, A, F, E, and the substitution is shown in Table 1.2.

Table 1.2 – Caesar cipher with $k_1 = 3$ and k_2 = CRIPTOGRAFIE

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	C	R	I	P	T	O	G	A	F	E	Ă	Â	B	D	H	Î
(x+3) mod 31	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
c	P	T	O	G	A	F	E	Ă	Â	B	D	H	Î	J	K	L

x	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
m	J	K	L	M	N	Q	S	Ș	Ț	U	V	W	X	Y	Z
(x+3) mod 31	19	20	21	22	23	24	25	26	27	28	29	30	0	1	2
c	M	N	Q	S	Ș	Ț	U	V	W	X	Y	Z	C	R	I

So CIFRULCEZAR is encrypted as POÂTXQPBIĂT, while the plain Caesar cipher with k = 3 gives FKITXOFHÂBT.

Key space. There are 31! orderings of the Romanian alphabet and 30 shifts, so the number of key pairs (k_1, k_2) is:

$$31! \times 30 = 246\,685\,159\,625\,337\,684\,531\,766\,886\,400\,000\,000 \approx 2.47 \times 10^{37}$$

This makes exhaustive search impractical, but the cipher is still monoalphabetic: every plaintext letter is always replaced by the same ciphertext letter, so it remains open to frequency analysis. Different key pairs can also produce the same substitution, so the number of distinct encryptions can never exceed 31!.

2. Implementation

The project has two files: `cipher.py` holds the cipher logic and the input validation, and `gui.py` holds the Tkinter window. The user selects the task (1.1 or 1.2) and the operation (encrypt or decrypt), enters the key(s) and the text, and presses the button. The window shows the normalized input, the result and the alphabet that was used (the permuted one for Task 1.2), so the result can be checked by hand. The main fragments are presented below.

2.1. Letter encoding

```
ALPHABET = ["A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I",
            "Î", "J", "K", "L", "M", "N", "O", "P", "Q", "R", "S",
            "Ș", "T", "Ț", "U", "V", "W", "X", "Y", "Z"]
N = len(ALPHABET)                       # 31
```

The alphabet is stored as a list in the exact order of Table 1.1, so the code of a letter is its index in the list. Built-in character encodings (ASCII or Unicode code points) are never used to compute the shift.

```
# Any accepted character -> its uppercase Romanian letter
TO_UPPER = {}
for up, low in zip(ALPHABET, LOWER):
    TO_UPPER[up] = up
    TO_UPPER[low] = up
# Cedilla variants (Ş ş Ţ ţ) are treated as comma-below letters (Ș ș Ț ț)
TO_UPPER.update({"Ş": "Ș", "ş": "Ș", "Ţ": "Ț", "ţ": "Ț"})
```

The `TO_UPPER` dictionary maps every accepted character, uppercase or lowercase, to its uppercase letter. The cedilla variants Ş, ş, Ţ, ţ are mapped to the comma-below letters Ș and Ț, so text typed on different keyboards is accepted.

2.2. Input validation and normalization

```
def normalize(text, allow_spaces=True):
    """Return the text in uppercase without spaces.
    Raises ValueError naming the first rejected character."""
    result = []
    for ch in text:
        if ch == " " and allow_spaces:
            continue
        if ch not in TO_UPPER:
            raise ValueError(f"Invalid character '{ch}'. Allowed: {ALLOWED_TEXT}.")
        result.append(TO_UPPER[ch])
    if not result:
        raise ValueError("The text must contain at least one letter.")
    return "".join(result)
```

`normalize()` removes the spaces, converts the text to uppercase and rejects any other character, naming it in the error message. An empty text is rejected too.

```
def parse_key(value):
    """Key 1: an integer from 1 to 30."""
    value = value.strip()
    if not (value.isdigit() or (value.startswith("-") and value[1:].isdigit())):
        raise ValueError(f"Invalid key '{value}'. The key must be an integer from 1 to
{N - 1}.")
    k = int(value)
    if not 1 <= k <= N - 1:
        raise ValueError(f"Invalid key {k}. The key must be an integer from 1 to {N -
1}.")
    return k
```

`parse_key()` accepts only an integer from 1 to 30. For anything else (0, 31, negative numbers, words) it raises an error that states the allowed range.

```
def parse_keyword(value):
    """Key 2: only Romanian letters, at least 7 characters."""
    value = value.strip()
    for ch in value:
        if ch not in TO_UPPER:
            raise ValueError(f"Invalid character '{ch}' in keyword. "
                             f"The keyword may contain only letters of the Romanian
alphabet.")
    if len(value) < 7:
        raise ValueError(f"The keyword is too short ({len(value)} characters). "
                         f"It must have at least 7 letters.")
    return "".join(TO_UPPER[ch] for ch in value)
```

`parse_keyword()` checks that the keyword has only Romanian letters and at least 7 characters, then converts it to uppercase.

2.3. Building the permuted alphabet

```
def permuted_alphabet(keyword):
    """Distinct keyword letters first, then the remaining letters in natural order."""
    result = []
    for ch in keyword + "".join(ALPHABET):
        if ch not in result:
            result.append(ch)
    return result
```

The keyword is joined with the full alphabet, and every letter is added to the result only the first time it appears. This puts the distinct keyword letters first and the remaining letters in their natural order, and always gives all 31 letters exactly once.

2.4. Encryption and decryption

```
def encrypt(text, k, alphabet=ALPHABET):
    index = {letter: i for i, letter in enumerate(alphabet)}
    return "".join(alphabet[(index[ch] + k) % N] for ch in text)


def decrypt(text, k, alphabet=ALPHABET):
    index = {letter: i for i, letter in enumerate(alphabet)}
    # Python's % is always non-negative; the extra + N keeps the formula
    # correct in languages whose remainder can be negative.
    return "".join(alphabet[(index[ch] - k + N) % N] for ch in text)
```

Both functions take the alphabet as a parameter: Task 1.1 uses the natural alphabet and Task 1.2 the permuted one, so the same code implements both ciphers. A dictionary gives the position of each letter, the shift is applied modulo 31 and the letter at the new position is taken. In decryption n is added before the modulo, so the formula would also be correct in languages where the remainder of a negative number is negative.

2.5. The graphical interface

```
    def _error(self, entry, message):
        """Show the message, mark the field and ask for the value again."""
        entry.config(bg=ERROR_BG)
        entry.focus_set()
        entry.select_range(0, "end")
        self.status.config(text="Error: " + str(message))
        self._set_readonly(self.result, "")
        self._set_readonly(self.norm_out, "")

    # ------------------------------------------------------------ action
    def run(self):
        for e in (self.key1, self.key2, self.text):
            if e.cget("state") == "normal":
                e.config(bg=OK_BG)
        try:
            k = parse_key(self.key1.get())
        except ValueError as err:
            return self._error(self.key1, err)

        alphabet = ALPHABET
        if self.task.get() == "2":
            try:
                keyword = parse_keyword(self.key2.get())
            except ValueError as err:
                return self._error(self.key2, err)
            alphabet = permuted_alphabet(keyword)
        self.show_alphabet(alphabet)

        try:
            text = normalize(self.text.get())
        except ValueError as err:
            return self._error(self.text, err)

        out = encrypt(text, k, alphabet) if self.op.get() == "E" else decrypt(text, k,
alphabet)
        self._set_readonly(self.norm_out, text)
        self._set_readonly(self.result, out)
        self.status.config(text=" ")
```

`run()` is called by the Encrypt/Decrypt button (or the Enter key). It validates the fields in order: key 1, then key 2 for Task 1.2, then the text. If a value is invalid, `_error()` shows the message in red, colours the wrong field and puts the cursor in it, so the user can correct the value and try again. Otherwise the result and the alphabet that was used are displayed.

3. Results

3.1. Task 1.1 – Caesar cipher

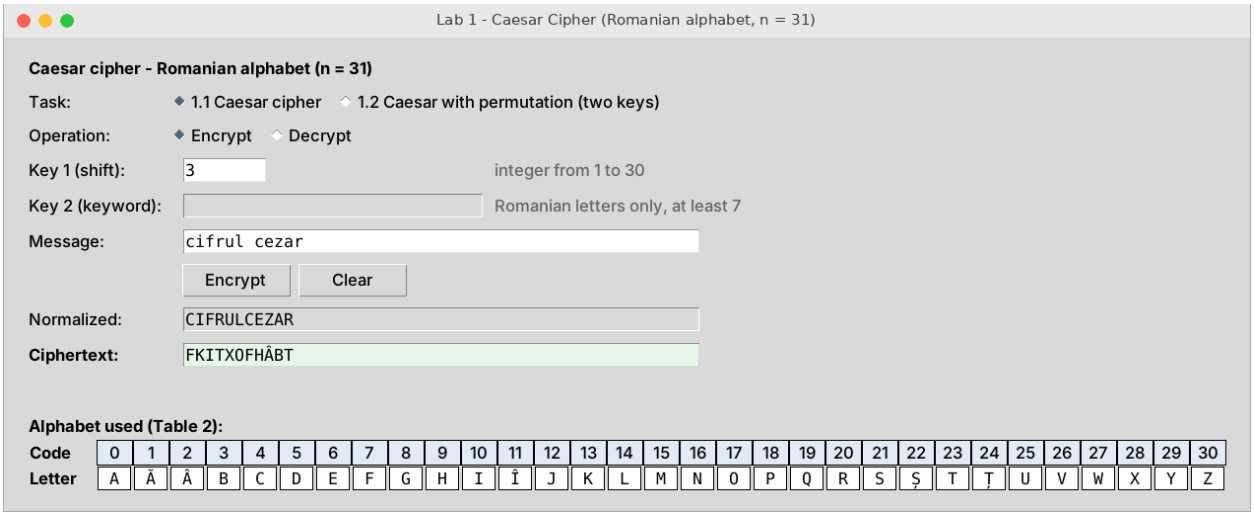


Figure 3.1 – Encryption of "cifrul cezar" with k = 3

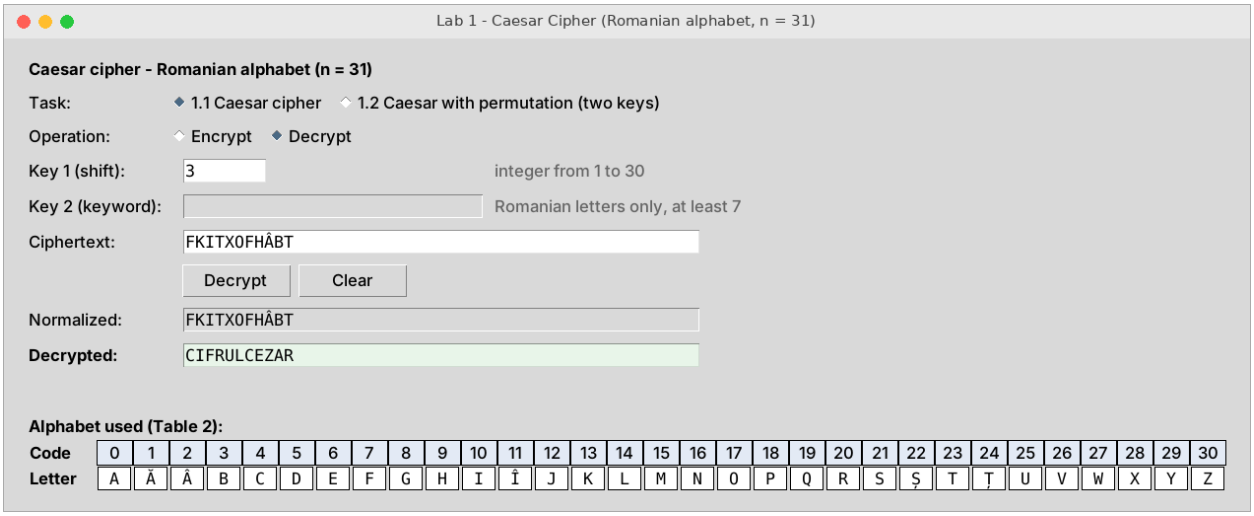


Figure 3.2 – Decryption of FKITXOFHÂBT with k = 3

Manual check: C (4) → 7 = F, I (10) → 13 = K, Z (30) → 33 mod 31 = 2 = Â, which matches the program output.

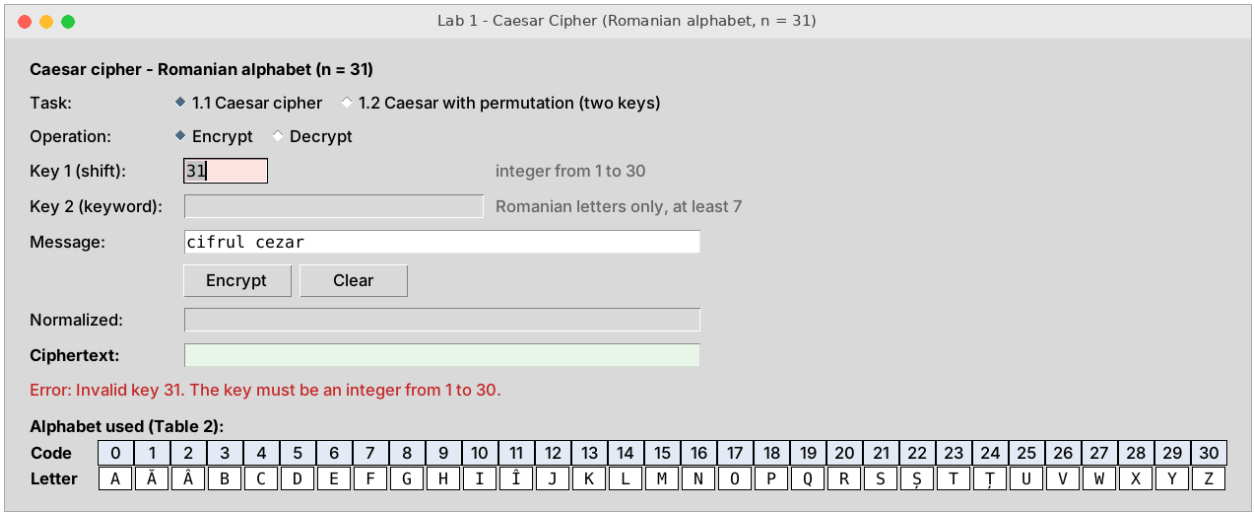


Figure 3.3 – Invalid key: 31 is outside the range 1…30

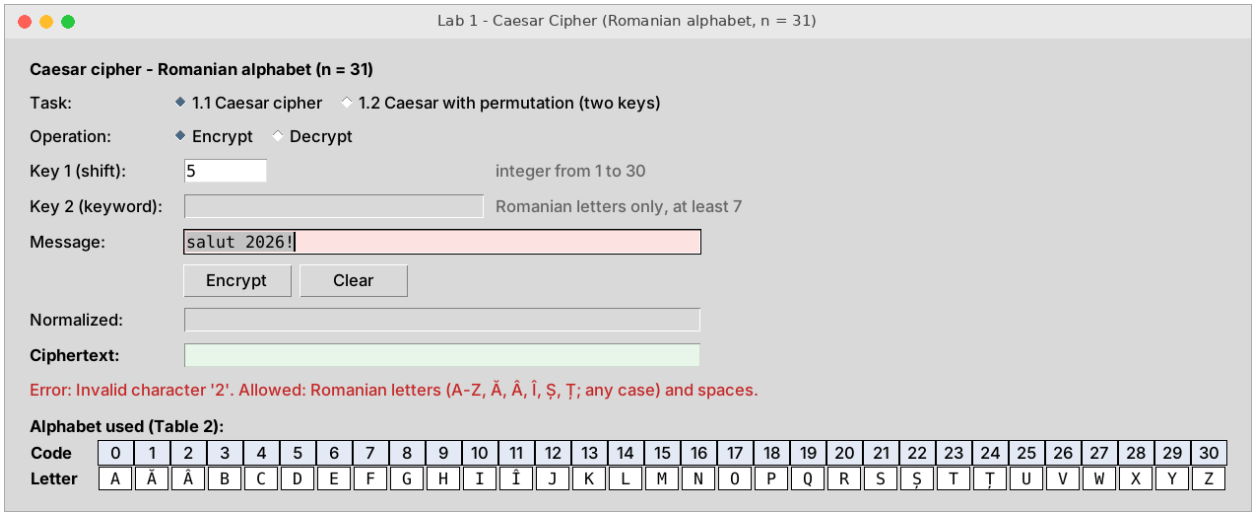


Figure 3.4 – Invalid character in the message

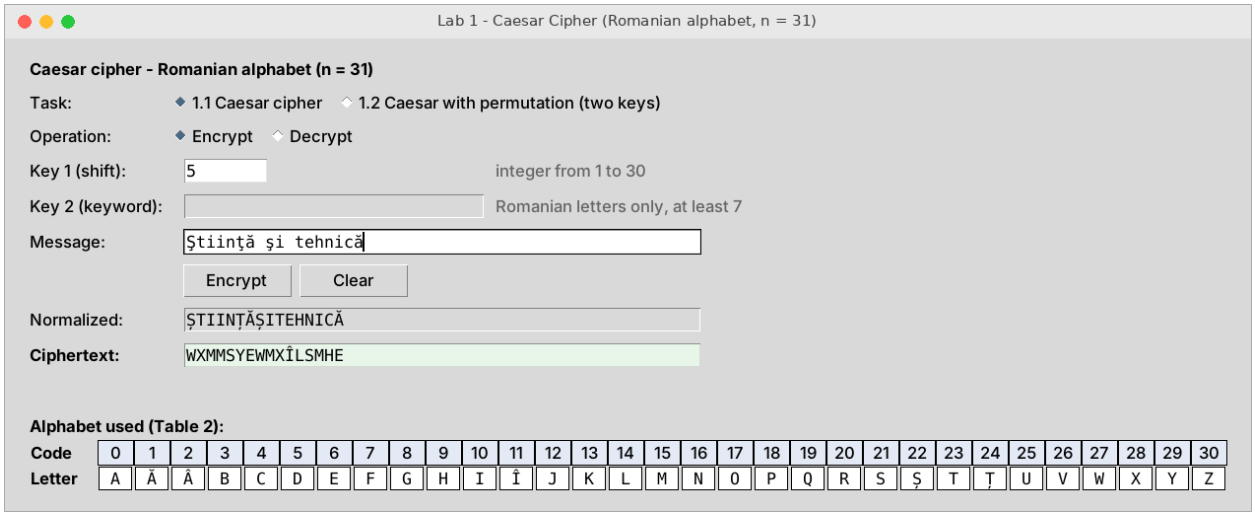


Figure 3.5 – Letters typed with a cedilla (Ş, ţ) are accepted as Ș and Ț

3.2. Task 1.2 – Caesar cipher with two keys

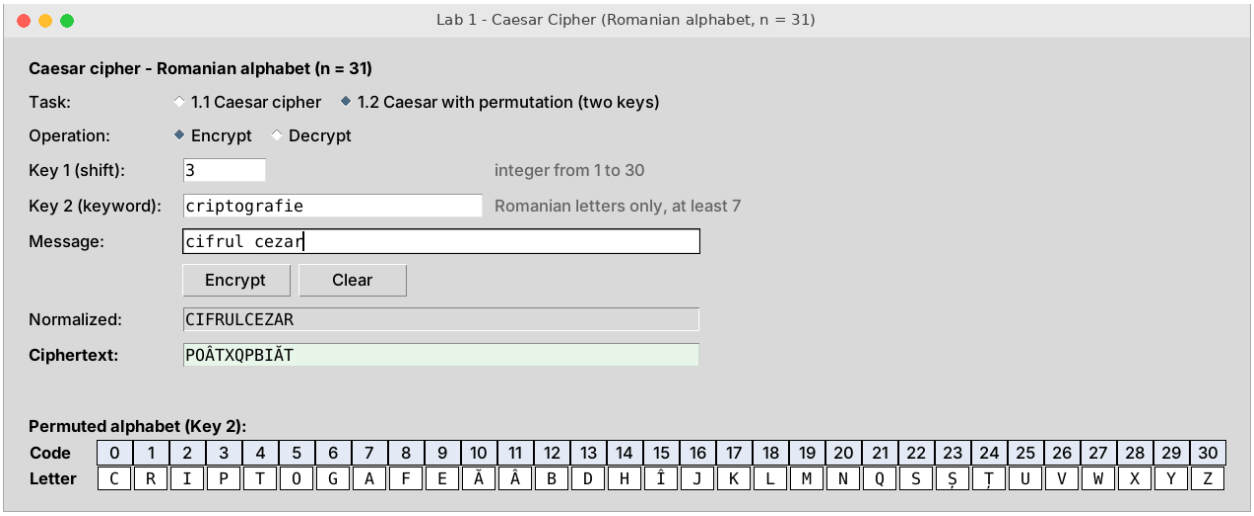


Figure 3.6 – Encryption with $k_1 = 3$ and k_2 = criptografie

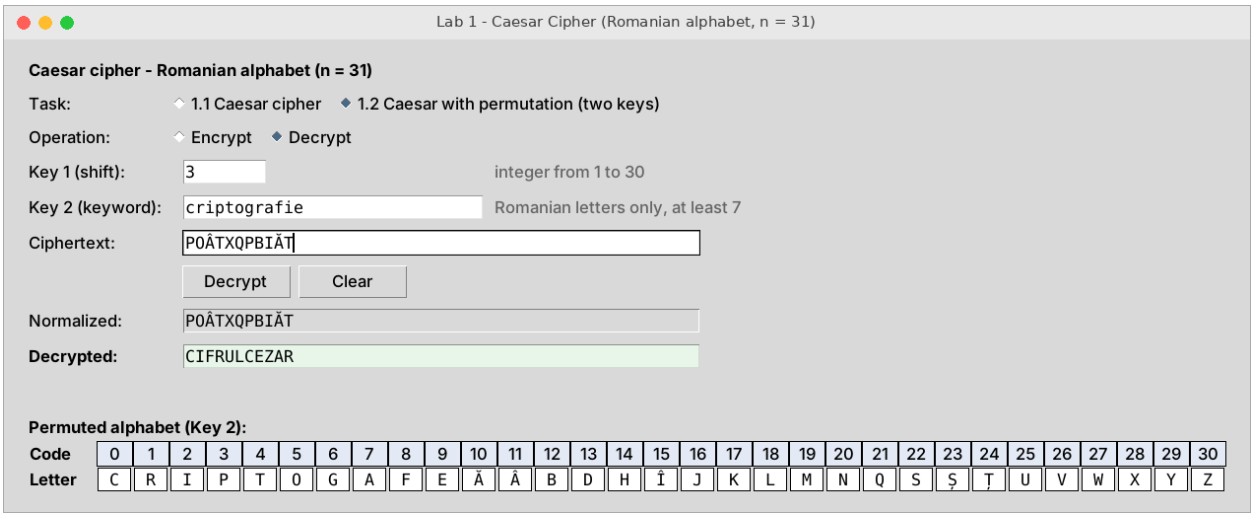


Figure 3.7 – Decryption of POÂTXQPBIĂT with the same keys

The permuted alphabet shown by the program is the same as in Table 1.2, and the ciphertext POÂTXQPBIĂT matches the manual calculation.

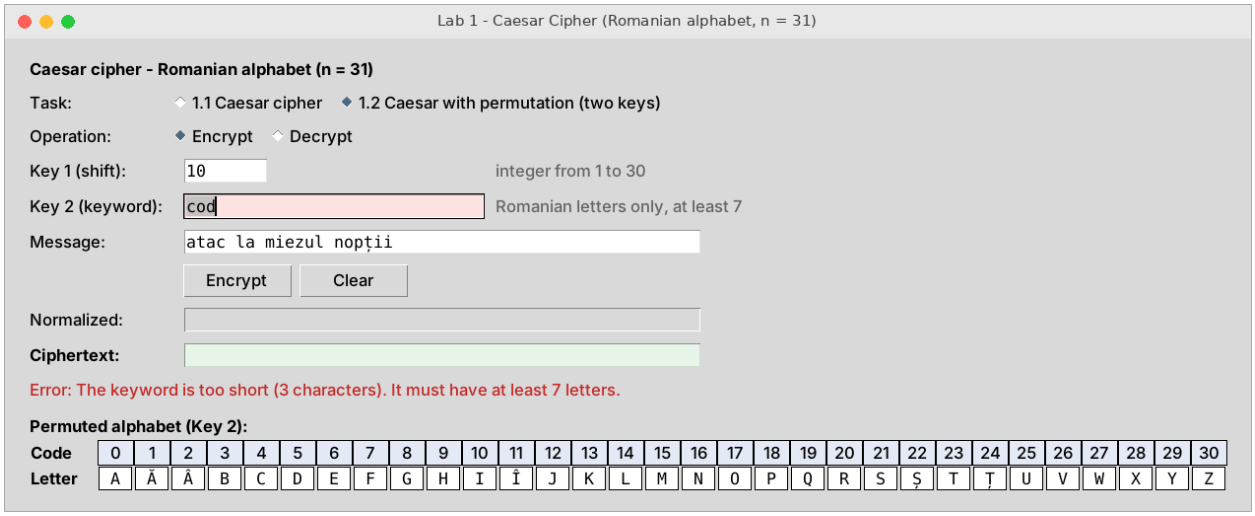


Figure 3.8 – Keyword that is too short

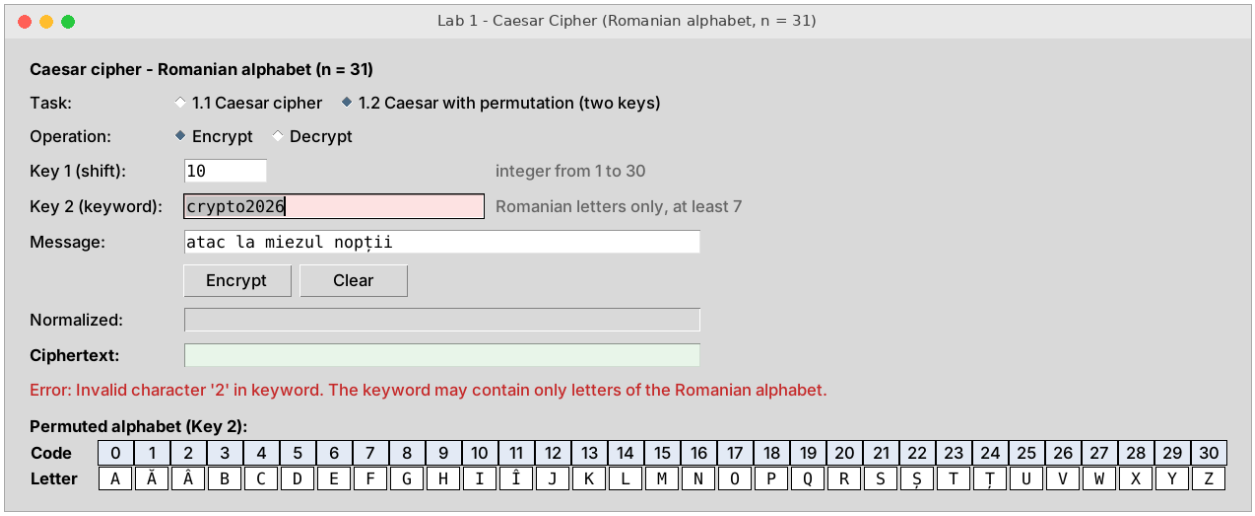


Figure 3.9 – Keyword that contains a digit

4. Conclusions

The classic Caesar cipher is very weak. Over the Romanian alphabet it has only 30 keys, so all of them can be tried by hand in a few minutes, and a computer does it instantly. It gives no real protection today.

The keyword permutation raises the key space to $31! \times 30 \approx 2.47 \times 10^{37}$ key pairs, which makes brute force infeasible. The real gain is smaller than this number suggests: the cipher is still a monoalphabetic substitution, so the letter frequencies of Romanian stay visible in the ciphertext and can be used for frequency analysis. In practice the keyword is usually a real word, which reduces the effective key space even more.

During this work I learned to model an alphabet as a list of codes instead of relying on character encodings, to handle modular arithmetic correctly for negative values, to build a keyword-

based permutation and to validate user input in a GUI, including the cedilla and comma-below forms of Ș and Ț.

5. Source code

The full source code is available on GitHub: https://github.com/Afyze/cs-2026-AugustinCarazanu/tree/main/lab1